Deep Unsupervised Learning

Mihir Rao

Contents

1 Introduction **2**
1.1 Learning from raw data 2
1.2 Patterns, prediction, and compression 2
1.3 Generating and conditioning 3
1.4 Compression and downstream tasks 3

Lecture 1: Introduction

1.1 Learning from raw data

Deep unsupervised learning uses deep networks to capture structure in raw data without requiring a human-provided label for each example. Two broad approaches are:

- Generative modeling: learn a model of a data distribution, so that it can produce new samples or assign information to observations.
- Self-supervised learning: construct a training task from the data itself, such as predicting a missing part from the visible part. The prediction task supplies its own targets and can encourage useful representations.

Both approaches can exploit large datasets that would be expensive to label for a specific application. Plausible samples need not match the true distribution, and a pretext task need not help every downstream task.

1.2 Patterns, prediction, and compression

A pattern permits a shorter description than a literal recording. For example, a predictable part of a sequence need not be encoded independently of its context. The connection between prediction and compression makes this precise for discrete data: a probability model q can be used to assign a code length of approximately $-\log_2 q(x)$ bits to outcome x. A more probable outcome gets a shorter code. If X is drawn from distribution p, the expected ideal length is

$$\mathbb{E}_{X\sim p}[-\log_2 q(X)] = H_2(p) + D_{\mathrm{KL},2}(p\|q),$$

where H_2 is entropy and the divergence measures the mismatch between the true distribution and the model, in bits. A well-matched model can therefore compress typical data efficiently; assigning too little probability to actual outcomes makes their descriptions costly.

At an idealized level, finding a short description of data is a way to express the discovery of regularities. Kolmogorov complexity asks for the length of a shortest program describing an object, and Solomonoff induction relates prediction to a preference for short descriptions. These are conceptual ideals, not practical claims that a trained network finds the shortest possible program or achieves optimal inference.

Pretraining can help a downstream task when the two datasets share relevant structure, but successful transfer is not automatic.

1.3 Generating and conditioning

A generative model can produce images, audio, text, video, or three-dimensional objects. In CONDITIONAL GENERATION, the output is guided by an input, such as generating an image from a text description, synthesizing speech from text, or transforming one kind of image into another. Conditioning narrows the desired output distribution, but many outputs may still be compatible with the same input. A generated continuation can be fluent without being factually correct, as illustrated by plausible but invented news-like text.

Models of video or other temporal data can also predict possible future observations. An interactive predictive model can be used as a simulator for control, provided its predictions are accurate enough for the decisions being made. Scientific applications include learning from large collections of biological data; protein structure prediction illustrates the value of learned representations for problems beyond natural images and language.

1.4 Compression and downstream tasks

LOSSLESS COMPRESSION recovers the original exactly, while LOSSY COMPRESSION trades some reconstruction error for a shorter representation. A learned distribution can support efficient lossless coding; a learned representation can also retain important visual structure at a constrained bitrate in lossy coding. The appropriate measure of error depends on the use: visual similarity and exact pixel equality are different goals.

A common workflow first trains on a broad source dataset and then adapts the model to a downstream task. Adaptation can mean training a task-specific head, fine-tuning some or all parameters, or specifying a task through a prompt. In ZERO-SHOT use, no labeled examples of the target task are supplied at adaptation time; in FEW-SHOT prompting, a small number of examples are included in the prompt. These terms describe the target-task supervision, not an absence of pretraining data.

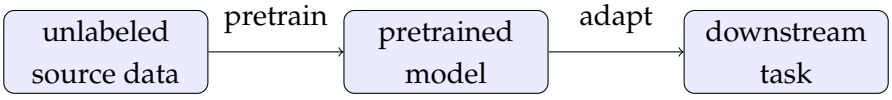


Figure 1: Pretraining reuses structure learned from data across tasks.

Language pretraining can support sentiment detection and other NLP tasks. In vision, contrastive learning and masked autoencoding can produce representations for recognition tasks. Their effectiveness depends on the data and training objective; scale and improvements in models and training have made these methods increasingly useful.

Source. Pieter Abbeel, Wilson Yan, Kevin Frans, and Philipp Wu, Deep Unsupervised Learning, Spring 2024, Lecture 1.